DEVBRIDGE – VERIFIED-TALENT FREELANCE PLATFORM

The Skill Graph

Module 6: Technical Report

Module 6 takes a verification report about one developer and produces a per-skill *belief*: not a rating, but a Gaussian over ability that widens when evidence is thin and widens again as evidence ages. This report names every model in the layer, states every free constant, and walks the scoring model through one real number, start to finish.

Author	*[Module 6 owner — fill in name / roll number]*
Module	M6 — Skill Graph (of six planned platform modules)
Contracts	`m6-parameters.yaml` v1.0.0, `ontology-seed.yaml` v0.1.0
Test status	562 tests passing, 0 failing
Date	September 13, 2026

Every figure in this report is reproduced from the committed source of Module 6, not quoted from a design document.

Contents

1 Before Anything Else: Three Refusals **2**

2 Architecture: Six Submodules, One Direction **2**

3 Model 1: The Event Projection Rule Table (6.0) **3**

4 Models 2 and 3: The Skill Graph (6.1) and the Absence of a Model (6.2) **4**

5 The Scoring Model (6.3) **5**

5.1 S1 — The Latent-Variable Model 5

5.2 S2 — The Prior 5

5.3 S3 — The Observation Model 6

5.4 S4 — The Update: a Closed-Form Kalman Step 7

5.5 S5 — The Decay Model 7

5.6 S6 — The Propagation Model 8

5.7 S7 — The Display Model: The Published Number is a Lower Bound 9

5.8 S8 — The Fold: How the Seven Parts Compose 9

6 Models 4 and 5: Ordering and Noticing **9**

7 End to End: One Real Number, All the Way Through **11**

8 Every Constant, in One Place **11**

9 Validation: Checking a Model That Was Never Trained **12**

10 Known Limitations and Open Work **13**

11 Conclusion **13**

1 Before Anything Else: Three Refusals

Module 6 is easiest to understand by what it declines to do. Each refusal below is enforced by code and held by a test suite, not written down and hoped for; the design of every later section follows from these three.

Nothing here is trained.

No model is fitted, no optimiser runs, no labelled data exists anywhere in the module. Every constant is declared in a contract file (`contracts/m6-parameters.yaml`) with a stated reason and a sensitivity-sweep range. The update rule is closed-form Kalman arithmetic — arithmetic a reviewer can check by hand, which is the point.

There is no overall score.

Ability is per-skill or it is nothing. The endpoint `GET /developers/{ref}/score` returns `400` with an explanation, permanently. A single aggregate number would collapse an interval-valued, per-skill, decaying belief into something that reads like a credit rating.

The language model never judges.

Upstream, in Module 5, a large language model normalises free-text CV content into structured claims. It never assigns a score, a weight, or a verdict. Everything downstream of a claim is deterministic arithmetic over declared constants.

2 Architecture: Six Submodules, One Direction

Evidence flows one way through the layer. The ontology (6.1) is consulted by three stages but never receives anything back from them. Integrity analytics (6.5) reads the evidence log and the scored states, and writes nothing at all – which is what lets it stay entirely outside the scoring path.

Submodule	Package	Responsibility
6.0	`m6_projection`	Verified artefacts → evidence events (the rule table, Section 3)
6.1	`m6_ontology`	Skills, volatility tiers, typed relations (Section 4)
6.2	`m6_ingestor`	Validation, consent, idempotency, append-only log (Section 4)
6.3	`m6_scoring`	The Bayesian filter: observation, update, decay, propagation (Section 5)
6.4	`m6_query`	Intervals, the `explain` endpoint, tiered search (Section 6)
6.5	`m6_integrity`	Findings about the shape of the evidence; changes no score (Section 6)

Table 1: The six submodules of Module 6 and the package each lives in.

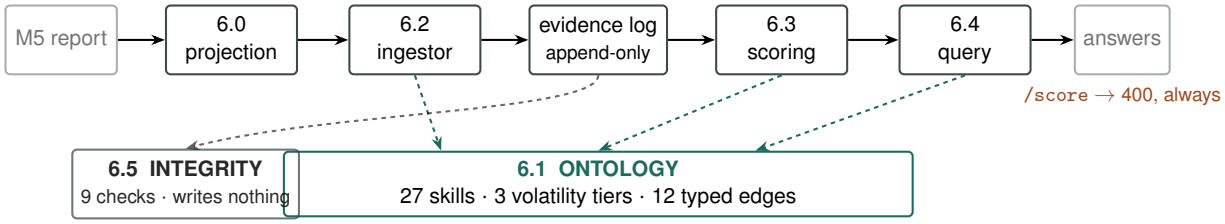


Figure 1: The one-way pipeline. The dashed teal arrows are the only thing 6.1 exposes: three read-only questions. 6.2 may ask `knows()` and nothing else – if the ingestor could reach the relation edges, propagation would migrate from read time (6.3, where it can be switched off and explained) to write time, where it cannot.

3 Model 1: The Event Projection Rule Table (6.0)

The first model in the layer is not statistical, and saying so plainly matters. It is a **declared lookup table** mapping each kind of verified artefact to three numbers: how strong a signal it is, how much the source can be trusted, and which skills it bears on. Nothing here is inferred; the table lives in `contracts/projection-rules.yaml` and a reviewer can disagree with any single row of it.

Evidence kind	Emitted when	strength	reliability	direction
`platform_rating`	handle resolves and a max rating exists	0.80	0.90	positive
`repo_artifact`	repo exists, owner matches, not a fork or shell	0.45+0.05·band	0.70	positive
`claim_corroborated`	a CV skill claim is matched by verified evidence	0.70	0.80	positive
`artifact_live`	a deployment responds	0.40	0.85	positive
`credential_verified`	the issuer confirms the credential	0.60	0.95	positive
`cv_claim`	a claim with no corroboration anywhere	0.30	0.20	positive
`claim_contradicted`	a decisive check fails against a skill claim	0.90	0.95	negative

Table 2: The seven emit rules. `strength` and `reliability` travel on the event and enter the scoring model's variance term (Section 5.3); nothing else about the artefact does.

Reliability factors, named on the event

Reliability is multiplied by discount factors, each recorded on the event by name, so a candidate who asks why a number is low gets the factor, not a total.

$$\text{reliability} = \text{reliability_base} \times \prod \text{factors}$$

Factor	Value	Applies to
`ownership_unverified`	$\times 0.6$	the GitHub identity is self-claimed, not proven (positive-direction events only)
`scraped_source`	$\times 0.7$	a profile page was parsed, not an API queried
`aggregate_across_sources`	$\times 0.7$	the claim pools a count no single source can confirm
`stale_evidence`	$\times 1.0$	deliberately inert — decay handles age at scoring time

Table 3: Reliability discount factors from `contracts/projection-rules.yaml`.

`ownership_unverified` applies only to evidence that would *raise* a score. A self-claimed account that contradicts the CV is credible — nobody volunteers a handle that undermines them — so the discount is asymmetric on purpose. `stale_evidence` exists and does nothing: applying an age discount here as well as in the decay kernel (Section 5.5) would count staleness twice, and the contract records the zero explicitly so nobody re-enables it later.

A rule with teeth

Every claim must end up either carried by an event or named in `not_emitted[]` with a reason. Silence is not an available outcome. This invariant is checked at projection time, and when it was added it immediately caught two claim kinds that had been vanishing without trace: a `certification` claim, and an uncorroborated `project` claim.

4 Models 2 and 3: The Skill Graph (6.1) and the Absence of a Model (6.2)

Model 2 — The Volatility-Tier Model

Every skill is filed in one of three tiers, and the tier — not the candidate, not the evidence — fixes how fast a signal about it goes stale. Three tiers rather than a continuum, because a continuum is a parameter per skill and cannot be defended or swept.

Tier	λ per day	$\tau = k/\lambda$	true half-life $\tau \ln 2$	Example skills
fundamental	1.0×10^{-4}	3500 d	2426 d	algorithms, C++, data structures
ecosystem	3.0×10^{-4}	1167 d	809 d	Python, TypeScript, Go, SQL
framework	7.5×10^{-4}	467 d	324 d	React, Docker, PostgreSQL

Table 4: The three volatility tiers, from `contracts/m6-parameters.yaml`.

Someone who could reason about algorithms in 2019 still can; a framework signal from three years ago genuinely means less. Getting a tier wrong is the difference between a skill that fades in a year and one that holds for a decade, so tiers are reviewed in a YAML file rather than set in code.

Naming discrepancy, flagged not hidden

The contract calls k/λ the half-life, and the API publishes it as `half_life_days`. Strictly it is the *e-folding time constant* τ: at $t = \tau$ the mean has moved 63.2% of the way back to the prior, not 50%. The true half-life is $\tau \ln 2$ — the fourth column above. The arithmetic is correct everywhere; only the label is loose, and the fix is a rename rather than a change of behaviour.

Model 3 — The Relation Model

Twelve directed, typed edges. An edge asserts that evidence for one skill is genuine evidence about another — a strong thing to claim, so the graph is deliberately sparse and every hop is

damped. The type carries the damping coefficient γ; there is one γ per *type*, never per edge, because per-edge coefficients are an unbounded parameter space nobody could sweep or defend.

Edge type	γ	Example
`PREREQUISITE_OF`	0.55	algorithms → problem_solving
`REQUIRES`	0.65	C++ → algorithms, Next.js → React
`RELATED_TO`	0.40	C++ → C (the weakest claim, most damped)

Table 5: The three edge types and their damping coefficients.

Direction matters: `RELATED_TO` reads symmetric and is not. Propagation only ever runs out of a skill that has direct evidence, so the arrow decides which way a signal can travel. A genuinely two-way relation is written as two edges.

No Model — 6.2, the Evidence Ingestor

Worth stating explicitly, because the gap is the design. The ingestor validates against the schema, refuses anything without consent scope `skill_graph_inclusion`, deduplicates by content hash, appends to a log guarded by SQL triggers that make `UPDATE` and `DELETE` raise, and quarantines skills the ontology does not know. It computes nothing. A quarantined skill is stored and flagged, never dropped — dropping it would lose evidence the ontology has merely not caught up with; scoring it would invent a node nobody defined.

5 The Scoring Model (6.3)

This is the model the rest of the layer exists to serve. It is a **Bayesian filter in logit space**: one Gaussian belief per (developer, skill) pair, updated in closed form by each piece of evidence and widened by the passage of time. It has eight separable components, and every one of them is arithmetic that can be checked on paper.

5.1 S1 — The Latent-Variable Model

Ability for one developer on one skill is a latent variable $\theta \in (0,1)$, and the filter does not act on θ directly. It acts on its logit, which is unbounded — and that is what makes the Gaussian update exact rather than approximate. Belief at any moment is two numbers and a date.

$$x = \text{logit}(\theta) = \ln\frac{\theta}{1-\theta} \qquad \text{the state, unbounded}$$

$$x \sim \mathcal{N}(\mu, \sigma^2) \text{ as of date } t \qquad \text{everything the system knows}$$

$$\hat{\theta} = \text{sigmoid}(x) \qquad \text{monotone, so bounds stay bounds}$$

Working in a bounded space would need a Beta or logit-normal approximation at every step and would smear the arithmetic. Working in logit space costs one transform in and one out, and buys exactness in between.

5.2 S2 — The Prior

$$\theta_0 = 0.25 \;\Rightarrow\; \mu_0 = \text{logit}(0.25) = -1.0986, \qquad \sigma_0 = 1.30 \;\Rightarrow\; \sigma_0^2 = 1.6900$$

With no evidence at all, the published score is $\text{sigmoid}(\mu_0 - \sigma_0) = 0.083$. A developer with no evidence is not an average developer; they are an *unknown* one. Centring the prior below the median is what stops silence from paying: a profile with nothing in it does not sit at 0.5 waiting to be mistaken for adequate. σ_0 is wide enough — roughly 0.08 to 0.55 in display space — that no single weak event can manufacture a score out of it.

5.3 S3 — The Observation Model

Every event yields two numbers: *what was observed*, and *how much to trust the observation*. They come from different places, and the separation is the single most load-bearing decision in the module.

Observed value — four measurement methods

Method	Used by	Mapping
`percentile_against_reference`	`platform_rating`	observed $= a + b \cdot \text{ECDF}(\text{rating})$, with $a = 0.50$, $b = 0.40$. The empirical CDF is built once from the Codeforces user list and stored as a lookup table — a for-loop, not a training run. The reference population is developers generally, not the self-selected cohort that competes; being rated at all clears the general floor, which is what a encodes.
`substance_band`	`repo_artifact`	Four ordinal bands with stated thresholds — trivial 0.35, small 0.50, substantial 0.62, large 0.66. No trustworthy reference distribution of repository substance exists, so bands with published cut-offs are the honest form.
`proficiency_band`	`cv_claim`	familiar 0.30, intermediate 0.45, proficient 0.60, expert 0.75, read off the enum the candidate already filled in.

flat constant	`claim_corroborated`
	`artifact_live`
	`credential_verified`
	`claim_contradicted`
		0.68, 0.58, 0.70, and 0.15 for a contradiction.

Table 6: The four observed-value methods and every band or constant they use.

Observation variance — where trust lives

$$\sigma^2_{\text{obs}} = \frac{S_0}{\text{reliability} \cdot \text{relevance} \cdot \text{strength}}, \qquad S_0 = 0.10$$

All three multipliers arrive on the event; S_0 is the only free scale constant in the whole observation model. High reliability, high relevance, and high strength together mean a small variance, which means a large update.

Relevance is epistemic, and belongs here rather than in the observed value. If a repository is 40% Go, that does not mean the developer wrote worse Go — it means the repository says less about their Go. Folding relevance into the observed value would encode the first reading and quietly punish people for working on mixed codebases. Folding it into the variance encodes the second: the same claim about ability, held with less confidence.

An uncorroborated CV claim is kept out of the way by exactly this mechanism. Its observed

value is a perfectly respectable 0.45; its reliability is 0.20, so its variance is large and it barely moves the mean. The system records that the person said it, and declines to be persuaded.

5.4 S4 — The Update: a Closed-Form Kalman Step

$$K = \frac{\sigma^2}{\sigma^2 + \sigma^2_{\text{obs}}} \qquad \text{the Kalman gain, } 0 \ldots 1$$

$$\mu' = \mu + K\big(\operatorname{logit}(\text{observed}) - \mu\big) \qquad \text{move } K \text{ of the way to the observation}$$

$$\sigma^{2\prime} = \sigma^2(1 - K) \qquad \text{confidence only ever increases here}$$

The gain is the whole story: it is the fraction of the way from the current belief toward the new observation, and it is set entirely by which of the two is more certain. Against the wide prior the first event carries an enormous gain; each later event carries less, because there is less left to learn. That ordering is asserted in the test suite — a reader who saw later evidence weighted more heavily than the first would be right to distrust the whole thing.

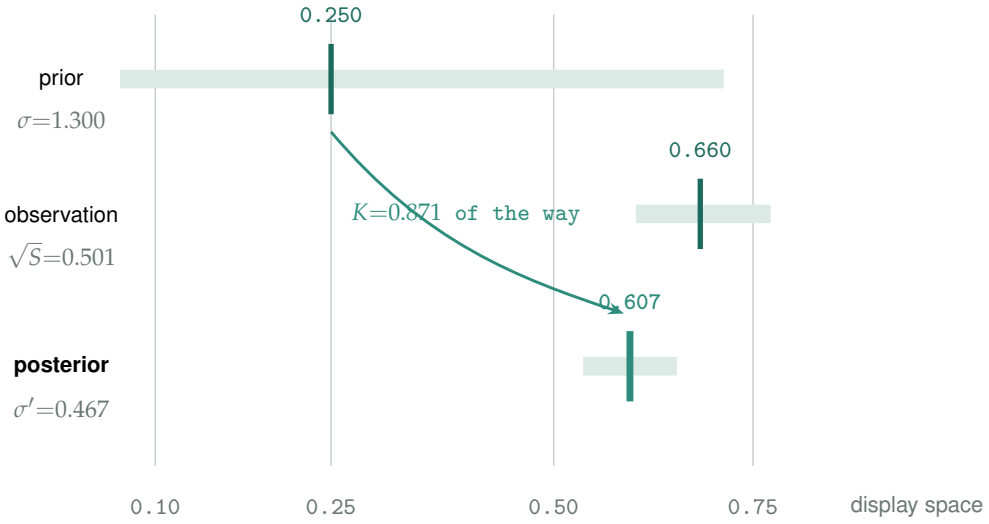


Figure 2: One update, to scale, on a shared logit axis. The prior's interval is so wide that a single moderately reliable observation pulls the belief 87% of the way across — and the posterior comes out narrower than *either* input, which is what a Bayesian update does. The axis is uniform in logit space, so the display-space ticks are unevenly spaced.

5.5 S5 — The Decay Model

Time is not evidence, so time must never sharpen a belief. Between observations the variance grows linearly and the mean reverts toward the prior, coupled so that one swept constant governs both instead of three independent knobs.

$$\sigma^2(t) = \sigma^2(\tau_{\text{last}}) + \lambda_s \, \Delta t \qquad \text{uncertainty grows, always}$$

$$\mu(t) = \mu + (\mu_0 - \mu)\left(1 - e^{-\Delta t \, \lambda_s / k}\right) \qquad \text{mean-reverts toward the prior}$$

$$k = 0.35, \qquad \tau_s = k/\lambda_s \qquad \text{reversion tied to volatility}$$

The consequence is worth stating in words, because it is the sentence a candidate deserves to hear: **a score falls with time because the system knows less, not because anyone decided the person got worse.** The mean drifts back toward "unknown" and the interval widens, and both effects are visible in the `explain` response as a day count and a half-life.

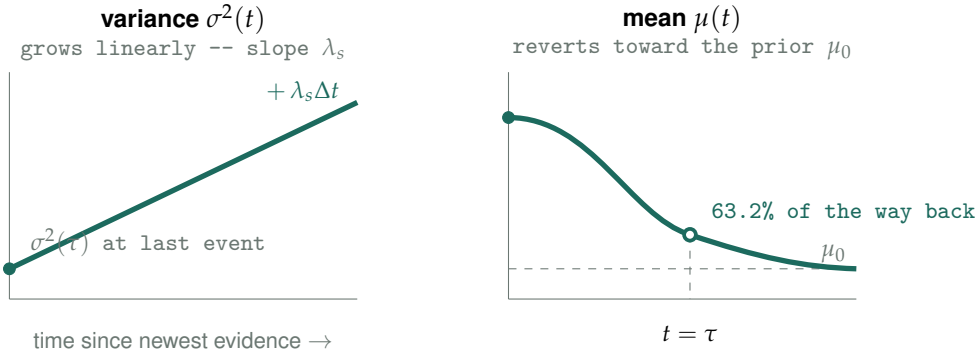


Figure 3: Two effects, one constant (schematic). Variance grows without bound while the mean asymptotes at the prior — so a very old belief converges on "we do not know," never on "this person is bad." Because $\tau = k/\lambda$, choosing a volatility tier sets both rates at once.

5.6 S6 — The Propagation Model

Evidence for C++ says something about C. Propagation is how that is expressed, and it is fenced in three ways: it runs at **read time** so nothing inferred is ever stored; it goes **one hop**; and it draws only from states that have **direct evidence**. Those three together make the pass non-iterative and loop-free without needing cycle detection at all.

$$\sigma^2_{\text{pseudo}} = \frac{\sigma^2_{\text{source}}}{\gamma} \qquad \text{then the ordinary S4 update, unchanged}$$

Damping reuses the observation mechanism instead of adding a second one. Weak evidence and distant evidence become the same kind of thing, which is precisely why they can share an update rule. An inferred skill is presented with `evidence_count: 0`, `basis: "propagated"`, and the edge it came along — never clamped, never disguised as measurement.

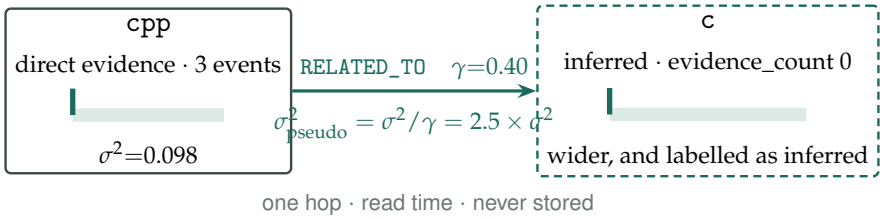


Figure 4: Damping is variance inflation. The weakest edge type divides confidence by the most: $\gamma = 0.40$ means the neighbour arrives $2.5\times$ less certain than it actually is. A skill that already has direct evidence is never overwritten by an inference — real evidence always wins.

5.7 S7 — The Display Model: The Published Number is a Lower Bound

$$\text{score} = \text{sigmoid}(\mu - z\sigma), \qquad z = 1.0$$

What a client sees is not the mean. It is the **lower end of a one-sigma interval**, and this single choice does most of the ethical work in the module. A developer with thin evidence and a developer with abundant evidence can share a mean; only the second gets a high published score, because only the second's interval has closed up. Uncertainty costs the candidate something, which is the only arrangement under which *gathering more evidence* is the way to raise a number.

sigmoid is monotone, so a bound in logit space is still a bound after the transform. The interval always travels beside the score in every response, so it cannot be mistaken for a point estimate.

5.8 S8 — The Fold: How the Seven Parts Compose

State is a *derived cache and never a source of truth*. Any score is recomputed from the append-only log on demand, in a total order.

$$\begin{aligned}
&\text{state} && \leftarrow \text{prior at the first event's date}\\
&\text{for each event bearing on this skill,} && \text{ordered by } (\text{occurred_at}, \text{event_id}) :\\
&\quad \text{state} && \leftarrow \text{decay}(\text{state} \rightarrow \text{event date})\\
&\quad \text{state} && \leftarrow \text{update}(\text{state}, \text{observed}, \sigma^2_{\text{obs}})\\
&\text{state} && \leftarrow \text{decay}(\text{state} \rightarrow \text{as_of})
\end{aligned}$$

Ordering by $(\text{occurred_at}, \text{event_id})$ is a total order, so two events on the same day still fold deterministically. Recomputing rather than caching is what makes three otherwise expensive things cheap: `as_of` queries against any past date, ablation (fold again with one event withheld, to show what it was worth), and the parameter sweep.

6 Models 4 and 5: Ordering and Noticing

Model 4 — The Partial-Order Ranking Model

Search does not return a ranking, because the evidence does not support one. Two candidates whose intervals overlap are *not distinguishable*, and printing them in an order would invent a

fact. So search returns **tiers**: within a tier, the API says plainly that it cannot separate them.

The caller also chooses the risk posture rather than having one chosen quietly for them: `definite=true` requires the whole interval to clear the bar; `definite=false` requires only that the interval reaches it.

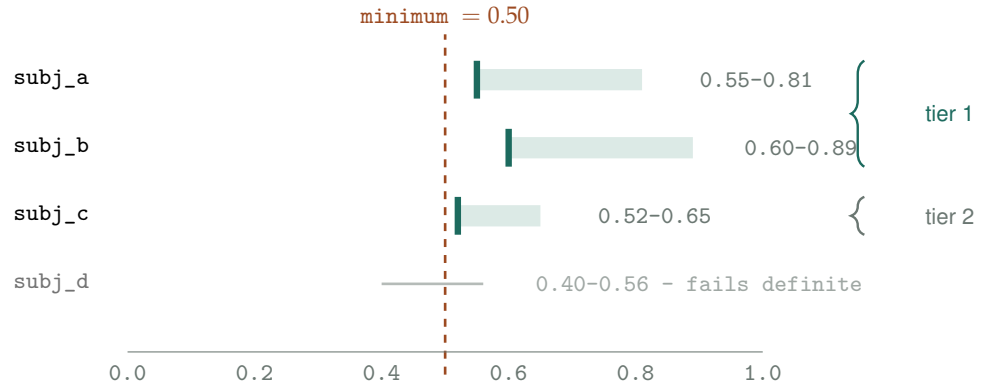


Figure 5: A partial order, not a ranking. `subj_a` and `subj_b` overlap, so they share a tier and no order is asserted between them. `subj_c` clears the bar in its own tier; `subj_d` reaches the threshold but does not clear it definitely, so a `definite` search excludes it. Measurement showed that separating two candidates whose means differ by 0.10 needs $z \leq 0.40$ — at the published $z = 1$ this search can filter but honestly cannot rank.

Model 5 — The Integrity Model: Nine Rules That Change No Number

The obvious design is to spot something suspicious and discount the evidence. That is exactly what this must not do: a signal that silently moves a score is an unexplainable penalty, and with no overall trust number there is nowhere for it to hide. So 6.5 produces **findings**, and a test asserts the skill graph is byte-identical before and after it runs.

Check	Severity	What it notices
`EVIDENCE_BURST`	attention	many artefacts appearing in a narrow window
`OWNERSHIP_UNPROVEN`	attention	the GitHub identity was never proven
`CONTRADICTED`	attention	a decisive check failed against a claim
`SINGLE_SOURCE`	notice	a skill resting entirely on one origin
`CLAIM_ONLY`	notice	a skill with nothing but self-report behind it
`INFERRED_ONLY`	notice	a skill reached only by propagation
`WHOLLY_STALE`	notice	all evidence older than the skill's own half-life
`BREADTH_FROM_ONE_ARTEFACT`	notice	many skills claimed off a single repository
`QUARANTINED_EVIDENCE`	notice	evidence held out because the ontology lacks the node

Table 7: The nine integrity checks and their severities. There is deliberately no third, more severe level.

Three properties are structural. Every finding **names its evidence**. Every finding **carries an innocent explanation** — a burst of recent commits is a student finishing coursework far more often than it is anything else. And there is **no "fail" severity**: two values only, because a third would invite somebody to threshold on it.

7 End to End: One Real Number, All the Way Through

Everything above, applied to the skill `go` for the candidate in the worked example. Two events, both dated 2026-06-10, queried 81 days later. These figures are the ones the shipped code emits.

Step	obs.	σ^2_{obs}	K	μ	σ	score	mean	upper
prior, no evidence	—	—	—	−1.0986	1.300	0.083	0.250	0.550
`repo_artifact`: distributed-kv, 168 commits, 92% Go	0.660	0.2506	0.871	+0.4357	0.467	0.492	0.607	0.712
`claim_corroborated`: the CV's Go claim, now backed	0.680	0.1786	0.550	+0.6107	0.313	0.574	0.648	0.716
decay to `as_of`, +81 days	—	—	—	+0.4960	0.350	**0.536**	0.622	0.700
projected, if nothing arrives for +1 year	—	—	—	+0.1515	0.456	0.425	0.538	0.647
projected, for a full τ (1167 days)	—	—	—	−0.4700	0.670	0.242	0.385	0.550

Table 8: The `go` skill through six fold steps. Reproduced by `make demo`; matches `demo/output/04-explain.json` exactly.

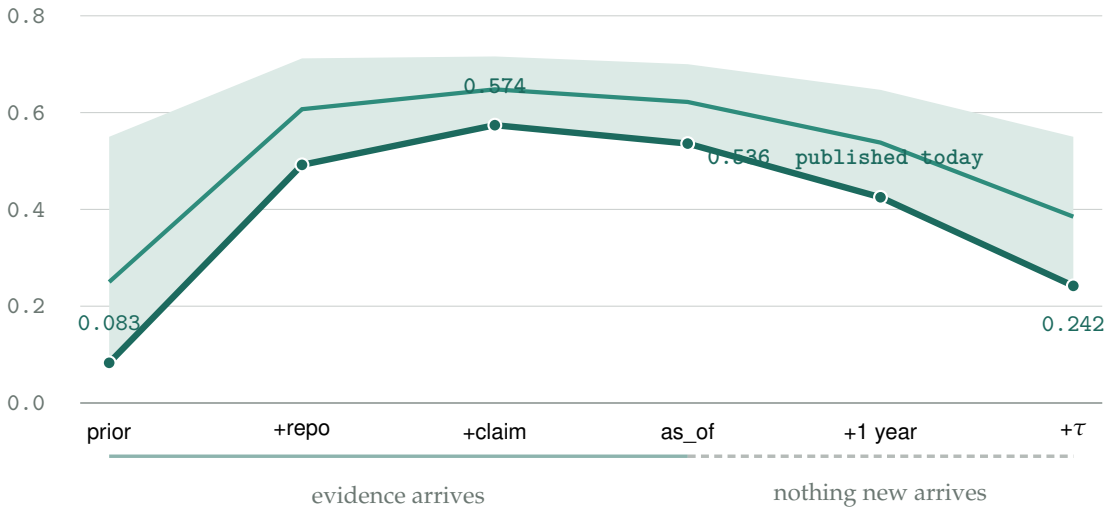


Figure 6: Evidence narrows the band; time widens it. The published score climbs from 0.083 to 0.574 not because the mean moved furthest — the mean only reaches 0.648 — but because the interval collapsed from 0.47 wide to 0.14. Then it reverses: with nothing new arriving, the band reopens and the number falls, having never claimed the person got worse.

8 Every Constant, in One Place

Code reads `value`; the sensitivity sweep reads `sweep`. Both come out of the same contract file, so no constant can drift from the one the sweep tested.

Symbol	Name	Value	Sweep range	Governs
θ_0	prior ability	0.25	0.15 – 0.40	where a developer with no evidence sits
σ_0	prior sigma	1.30	0.9 – 2.0	how easily the first event moves the belief
S_0	observation scale	0.10	0.05 – 0.30	the only free scale in the observation model

Symbol	Name	Value	Sweep range	Governs
z	display bound	1.0	0.5 – 2.0	how much uncertainty costs the candidate
k	half-life ratio	0.35	0.15 – 0.70	ties mean-reversion to volatility
λ fundamental	decay rate	1.0×10^{-4}	3.0×10^{-5} – 3.0×10^{-4}	algorithms, C++, data structures
λ ecosystem	decay rate	3.0×10^{-4}	1.0×10^{-4} – 8.0×10^{-4}	Python, TypeScript, Go
λ framework	decay rate	7.5×10^{-4}	3.0×10^{-4} – 2.0×10^{-3}	React, Docker, PostgreSQL
γ `PREREQUISITE_OF`	damping	0.55	0.3 – 0.8	propagation across a prerequisite edge
γ `REQUIRES`	damping	0.65	0.4 – 0.9	propagation across a requires edge
γ `RELATED_TO`	damping	0.40	0.2 – 0.6	propagation across a related edge
a, b	ECDF affine map	0.50, 0.40	—	rating percentile $\rightarrow$ observed ability
depth	propagation hops	1	fixed	never more than one hop, ever

Table 9: All fourteen free constants declared in `contracts/m6-parameters.yaml`. None is fitted.

Known limitation, stated up front

The Codeforces ECDF is currently *approximated* rather than regenerated from the real user list — no network access was available in the build environment. Every `platform_rating` number is therefore provisional until that table is rebuilt. The contract records this as `status: approximated_pending_regeneration`, so the caveat travels with the data rather than living in someone's memory.

9 Validation: Checking a Model That Was Never Trained

With no labelled data there is no test-set accuracy to quote, so the model is checked against a property it must have rather than a benchmark it should beat: **if a true ability is known, does the filter recover it, and is the filter's own stated uncertainty honest?** Three hundred synthetic developers with known ability, twelve events each, folded through the shipped code.

Metric	Measured	Target
Bias (mean estimate − mean truth)	−0.0236	$\lvert\text{bias}\rvert < 0.05$
RMSE	0.0510	< 0.15
Coverage (truth inside $\mu \pm z\sigma$)	0.677	nominal (Gaussian) $= 0.682$
Mean σ (logit space)	0.2376	—
Worst-case σ	0.1714	—

Table 10: The recovery gate: `python3 -m m6_scoring recover`. **PASS.**

The coverage row is the result worth pausing on. A one-sigma interval around a Gaussian should contain the truth 68.2% of the time. This filter's intervals contain it 67.7% of the time — **the model's stated uncertainty matches its actual uncertainty**, with nothing tuned to make that happen. An overconfident filter would score well on RMSE and fail here, which is exactly why the gate checks coverage and not only error.



Figure 7: More evidence, less error — monotonically. Not a benchmark score: a property the filter has to have. If error ever *rose* with more evidence, the update rule would be wrong, and no amount of tuning elsewhere would fix it.

What the Test Suite Actually Holds

562 tests, no external test framework required. The interesting ones are not the assertions that a number equals a number — they are the ones that would fail if a stated principle stopped being true: that a suppressed claim is still reported to the candidate; that the graph is byte-identical before and after integrity analysis; that one developer's evidence never folds into another's; that a quarantined skill never reaches a score.

The models themselves were audited by **mutation testing**: make a submodule lie, run all 562 tests, and see who notices. That found a real gap. Making the ontology forget every skill's tier broke only two tests, one of which was the ontology testing itself — the scoring engine has a fallback tier list, so a broken ontology produced *plausible numbers computed from the wrong half-life*, the worst possible failure shape, with nothing positioned to doubt them. There is now a test that folds identical evidence under two ontologies differing only in one skill's tier, and asserts the score, the variance, and the interval width all move.

10 Known Limitations and Open Work

The ECDF.
Regenerate the Codeforces reference distribution from the real user list. Blocks publishing any `platform_rating` number as final.

The sensitivity sweep.
Run the full sweep and the ablation table across all fourteen constants. The machinery exists — `make sweep` enumerates every swept parameter — but the results are not yet written up.

The τ rename.
Rename `half_life_days` to reflect that it is the e-folding time constant, or publish $\tau \ln 2$ under the current name. Arithmetic unaffected either way.

11 Conclusion

Module 6 answers one question per skill — what does the evidence support, and how sure is that — and refuses to answer the question nobody can honestly answer, which is how good is this developer overall. Every number in it traces to a declared constant, a closed-form update, or a lookup table a reviewer can read without running any code, and the recovery gate shows that the filter's stated uncertainty is not merely plausible but measured to be accurate. That is the whole design: not a model trained to be right, but a model built so that being wrong would

be visible.

DevBridge Module 6 — skill graph layer — six packages, 562 tests passing. Every figure in this report was reproduced from the committed code on September 13, 2026.